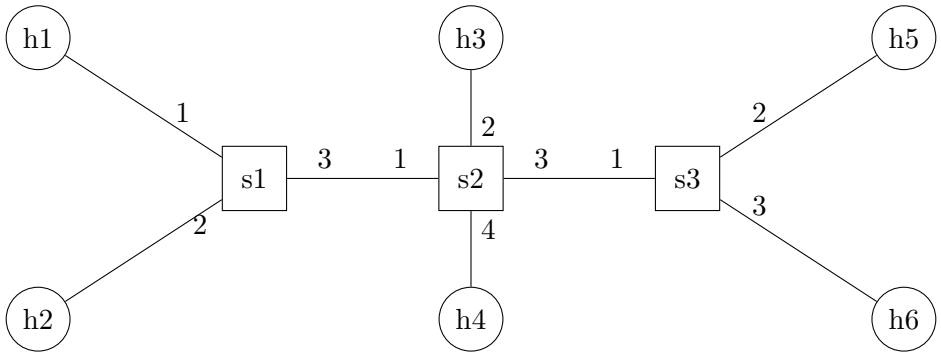
Software Defined Networking
17/06/2020

Gli studenti che usufruiscono della legge 170/2010 non rispondono alle domande indicate con la stella $\star$.

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm. La funzione di gestione dell'evento PacketIn è data nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Negli switch sono preinstallate le regole in Figura 1.

L'host h1 porta 1234 apre una connessione TCP verso h5 porta 5678.

1. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: S1 non invia messaggi al controllore.

2. Scrivere tutti gli host che ricevono il pacchetto. Per ogni host scrivere il TTL del pacchetto.

   Soluzione: h2: 200; h3, h4, h5: 199

3. Scrivere tutti gli host che ricevono la risposta di h5.

   Soluzione: h6, h3, h4, h1

L'host h1 porta 3322 invia un datagramma UDP verso h3 porta 123.

4. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

Soluzione: s1 invia una packet-in; riceve in risposta una flow-mod e una packet-out

5. Scrivere tutti gli host che ricevono il pacchetto. Per ogni host scrivere il numero di porta destinazione.

Soluzione: h2, h3, h4, h5, h6: 123

6. h3 invia una risposta. Scrivere il numero di porta destinazione con cui la risposta arriva in h1.

Soluzione: 3322

L'host h2 porta 2233 invia un datagramma UDP verso h3 porta 123.

7. ⋆ Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

Soluzione: S1 non invia messaggi al controllore.

8. ⋆ Scrivere tutti gli host che ricevono il pacchetto. Per ogni host scrivere il numero di porta destinazione.

Soluzione: h2, h3, h4, h5, h6: 124

In s1 il pacchetto trova la regola lasciata dal pacchetto UDP del punto 4. In s2 la stessa regola non si applica perché il numero di porta è cambiato.

9. ⋆ h3 invia una risposta. Scrivere il numero di porta destinazione con cui la risposta arriva in h1.

Soluzione: Il pacchetto non arriva in h1. (in h2 arriva con numero di porta 2233)

Switch s1

Priority	Match	Action
5	*	Output(FLOOD)
10	eth_type = IPv4 ip_proto=UDP	Output(CONTROLLER)
15	eth_dst = MAC2	Output(2)
15	eth_dst = MAC1	Output(1)

Switch s2

Priority	Match	Action
0	*	Output(FLOOD)
10	eth_type = IPv4 ip_proto = TCP	DecTTL Output(FLOOD)
10	eth_type = IPv4 ip_proto = UDP	Output(CONTROLLER)

Switch s3

Priority	Match	Action
0	*	Output(FLOOD)
15	eth_dst = MAC5	Output(2)
15	eth_dst = MAC6	Output(3)

Figura 1: Regole iniziali

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
    buffer_id = msg.buffer_id; data = msg.data

    if (pkt_udp is not None):
        match = parser.OFPMatch(eth_dst=pkt_eth.dst, eth_type=ETH_TYPE_IP,
            ipv4_dst = pkt_ipv4.dst, ip_proto=IPPROTO_UDP,
            udp_dst=pkt_udp.dst_port)
        actions = [
            parser.OFPSetField(udp_dst = pkt_udp.dst_port + 1),
            parser.OFPActionOutput(FLOOD) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=20,
            match=match, instructions=inst)
        datapath.send_msg(mod)

    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
        in_port=in_port, actions=actions)
    datapath.send_msg(out)
```

Esercizio 2

Una matrice di commutazione crossbar di uno switch 4×4 usa l'algoritmo Take-a-Ticket per lo scheduling delle trasmissioni. Si assumano pacchetti tutti di lunghezza L,

Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto.

Ingresso 1: 4, 3, 1

Ingresso 2: 1, 3, 2

Ingresso 3: 1, 4, 2

Ingresso 4: 2, 4, 1

L'algoritmo di assegnazione dei ticket privilegia le interfacce di ingresso secondo la seguente regola:

- le interfacce di uscita con identificativo dispari privilegiano le interfacce di ingresso con l'identificativo minore
- le interfacce di uscita con identificativo pari privilegiano le interfacce di ingresso con l'identificativo maggiore

10. Scrivere quali ticket sono assegnati dall'interfaccia di uscita 1 durante il primo round. Usare la notazione (i,j), dove i è il numero dell'interfaccia di uscita e j il numero progressivo del ticket.

    Soluzione: ingresso 1: (1, 1) e ingresso 4: (1,2)

11. Scrivere quali interfacce di ingresso trasmettono nel primo round.

    Soluzione: 1 e 3

12. Scrivere quali interfacce di ingresso trasmettono nel secondo round.

    Soluzione: 1, 2, 3, 4

13. Scrivere il numero totale di pacchetti trasmessi nei primi tre round.

    Soluzione: 9

Si consideri ora uno scenario aternativo, in cui i pacchetti in entrata dall'ingresso 3 sono lunghi $2L$.

14. $\star$ Scrivere quali interfacce di ingresso trasmettono un nuovo pacchetto nel secondo round.

Soluzione: 1 e 4

15. ⋆ Scrivere il numero totale di pacchetti trasmessi nei primi tre round.

Soluzione: 6

Esercizio 3

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Protocol
80.0.0.0/5	*	80	*	TCP
80.0.0.0/4	128.0.0.0/2	80	22	TCP
*	96.0.0.0/3	*	53	UDP
88.0.0.0/5	*	*	123	UDP
92.0.0.0/6	64.0.0.0/2	23	*	UDP
92.0.0.0/7	64.0.0.0/3	25	80	*

16. Scrivere il bitvector per la entry 88.0.0.0/5 del campo Source Address.

Soluzione: 011100

17. Scrivere il bitvector per la entry 92.0.0.0/6 del campo Source Address.

Soluzione: 011110

La ricerca nel campo Destination Address è implementata con un algoritmo di ricerca binaria sulla lunghezza dei prefissi.

18. Scrivere se sono presenti marker e a quali lunghezze.

Soluzione: Non sono presenti marker

19. Scrivere la lista delle lunghezze di prefisso visitate dall'algoritmo per trovare un match per l'indirizzo 96.102.11.3.

Soluzione: 16, 8, 4, 2, 3